PROSIT 3

Groupe :

CARVAL Mathéo - Animateur

LUU Philippe - Scribe

BOUHAMED Allan - Secrétaire

CHALLIES Maeva - Gestionnaire du temps

Sommaire :

Sommaire :.....2

1. Contexte.....3
2. Mots inconnus / Notions à maîtriser.....3
3. Problématique.....3
4. Plan d'action (Estimé à 5h pour la phase d'étude).....4
5. Réalisation.....5

1. Contexte

Le projet CHU est dans les temps. Une réunion avec un panel d'utilisateurs (médecins, infirmiers, chercheurs, administrateurs) a revalidé toutes les demandes d'analyse : le modèle conceptuel ne change pas et l'ETL ne doit plus calculer que des agrégations. L'équipe lance donc l'implémentation du modèle physique de la base décisionnelle, son remplissage et les requêtes d'analyse, en s'appuyant sur un SGBD NoSQL comme Hive et son partitionnement. À plus long terme, les données sources seront mises à jour tous les 3 mois, au même rythme que la définition de nouvelles analyses. Surtout, un temps de réponse maximal de 2 minutes a été fixé, alors que des requêtes à plus de 3 jointures sur des millions de lignes peuvent largement dépasser ce seuil sans optimisation (partitionnement, bucketing, vues matérialisées). Il reste 3 jours pour sortir les analyses.

2. Mots inconnus / Notions à maîtriser

- **NoSQL (Not Only SQL)** : Famille de bases de données non relationnelles, conçues pour la distribution et les gros volumes.
- **Clé-valeur / Document / Colonnes / Graphe** : Les quatre familles NoSQL, selon la façon dont elles organisent les données.
- **ACID** : Propriétés des transactions des SGBDR : Atomicité (Tout ou rien : soit toutes les opérations passent, soit aucune), Cohérence (La transaction fait passer la base d'un état valide à un autre état valide, en respectant les contraintes), Isolation (Les transactions qui tournent en même temps ne se voient pas à moitié faites), Durabilité (Une fois validée (commit), la modification survit à une panne).
- **BASE** : Approche des bases distribuées : Basically Available (Le système répond toujours, même si un nœud est en panne, quitte à renvoyer une donnée pas à jour), Soft state (L'état peut changer tout seul, sans nouvelle écriture, le temps que les copies se synchronisent), Eventually consistent (Cohérence “ à terme” : si plus personne n'écrit, toutes les copies finissent par avoir la même valeur, mais pas instantanément.)
- **Théorème CAP** : Un système distribué ne peut garantir à la fois Cohérence, Disponibilité (Chaque requête reçoit une réponse, jamais d'erreur ni d'attente infinie, même si des nœuds sont en panne) et tolérance au Partitionnement réseau (Le système continue de fonctionner même si le réseau est coupé entre des serveurs, qui ne peuvent) : il en choisit deux (CA = un serveur unique, CP = renvoie une erreur plutôt qu’une fausse donnée, AP = répond une donnée possiblement fausse).
- **Partitionnement (Hive)** : Découpage physique d'une table en sous-dossiers selon une colonne (ex. année) pour ne lire que les données utiles.
- **Bucketing** : Répartition des lignes dans un nombre fixe de fichiers selon un hachage de colonne, pour accélérer jointures et échantillonnages.
- **Vue matérialisée** : Résultat d'une requête stocké physiquement et réutilisé au lieu d'être recalculé.
- **Agrégation** : Calcul d'une valeur de synthèse sur un groupe de lignes (COUNT, SUM, AVG).

- **Moteur d'exécution (MapReduce, Tez, Spark)** : Moteur qui exécute réellement les requêtes HiveQL sur le cluster.
- **Format colonnaire (ORC / Parquet)** : Stockage par colonne plutôt que par ligne, compressé, qui ne lit que les colonnes demandées.

3. Problématique

Comment choisir et optimiser le stockage physique de l'entrepôt (type de base NoSQL, partitionnement, bucketing, vues matérialisées) pour que des analyses portant sur des millions de lignes répondent en moins de 2 minutes, tout en supportant une mise à jour trimestrielle des données ?

4. Plan d'action (Estimé à 5h pour la phase d'étude)

1. Étude des familles de bases NoSQL, de leur organisation et de leurs cas d'usage (1h).
2. Comparaison SGBDR / NoSQL : propriétés ACID vs BASE et théorème CAP (0h45).
3. Sélection du type de stockage adapté au besoin du CHU et chargement via l'ETL (0h45).
4. Extraction d'informations avec MapReduce (0h45).
5. Optimisation du modèle physique (partitionnement, bucketing, vues matérialisées) et mesure des performances (1h45).

5. Réalisation

- **5.1 Familles de bases NoSQL (1h)** : Les bases NoSQL abandonnent le schéma relationnel rigide pour mieux se distribuer. Elles se classent en quatre familles :

Type	Organisation	Exemples	Cas d'usage	Intérêt pour le CHU
Clé-valeur	Une clé unique → une valeur opaque	Redis, DynamoDB	Cache, sessions, paniers	Cache des résultats de l'API
Document	Documents JSON au schéma souple	MongoDB, CouchDB	Catalogues, contenus variés	Dossier patient complet (hors analytique)
Colonnes	Familles de colonnes, stockage par colonne	HBase, Cassandra	Gros volumes, séries temporelles	Lecture rapide de quelques colonnes sur des millions de lignes
Graphe	Nœuds et relations	Neo4j	Réseaux sociaux, recommandations	Parcours de soins patient → médecin → établissement

  Hive n'est pas une base NoSQL au sens strict : c'est un entrepôt « SQL sur Hadoop » qui stocke ses tables en fichiers colonnaires (ORC, Parquet) sur HDFS. Il hérite de leurs avantages (distribution, volume) tout en gardant le SQL.
- **5.2 SGBDR vs NoSQL : ACID, BASE et CAP (0h45)** : Les deux mondes font des compromis différents :

	SGBDR (ACID)	NoSQL distribué (BASE)
Cohérence	Immédiate : tout le monde voit la même valeur	À terme : les copies finissent par converger
Transactions	Complètes, multi-tables	Limitées, souvent à un seul enregistrement
Schéma	Fixe, défini à l'avance	Souple
Montée en charge	Verticale (un plus gros serveur)	Horizontale (plus de serveurs)
Point fort	Fiabilité des écritures	Disponibilité et volume

  Théorème CAP : en cas de coupure réseau, il faut choisir entre cohérence et disponibilité.
  - **CP** (cohérence + partition) : HBase, MongoDB. On refuse de répondre plutôt que de donner une valeur fausse.
  - **AP** (disponibilité + partition) : Cassandra, CouchDB. On répond toujours, quitte à être en retard.
  - **CA** : un SGBDR sur un seul serveur, qui ne gère donc pas la partition.

  Pour le CHU, un indicateur de santé faux est pire qu'un indicateur en retard : la **cohérence prime**. Delta Lake apporte justement des transactions ACID au-dessus du stockage objet.
- **5.3 Choix du stockage et chargement (0h45)** : Le besoin est analytique : beaucoup de lectures agrégées sur de gros volumes, très peu d'écritures (une mise à jour tous

les 3 mois). Une base clé-valeur ou graphe ne sait pas faire de GROUP BY efficace ; le bon choix est un **stockage colonnaire interrogé en SQL distribué**.

Besoin	Choix retenu
Agrégats sur des millions de lignes	Format colonnaire Parquet (sous Delta Lake)
SQL pour les analyses	Catalogue Hive Metastore + moteur Trino
Mise à jour trimestrielle sans tout recharger	MERGE incrémental, réécriture des seules partitions modifiées
Cohérence	Transactions ACID Delta, retour à une version précédente (time travel)
Nouvelles analyses tous les 3 mois	Nouvelles requêtes sur le même modèle, sans changer le schéma

L'ETL ne calcule plus que les agrégations : âge et tranche d'âge, durée de séjour et de consultation, compteurs (nb_consultation, nb_sejour, nb_deces) et validité des scores de satisfaction (au moins 30 répondants).

- **5.4 Extraction d'informations avec MapReduce (0h45)** : Exemple : nombre d'hospitalisations par sexe et tranche d'âge (indicateur 5).

```
def map(sejour):                        # exécuté en parallèle sur chaque bloc
    yield (sejour.sexe, sejour.tranche_age), 1

def reduce(cle, valeurs):               # après le shuffle, une clé = un groupe
    yield cle, sum(valeurs)             # ('F', '60-69') -> nb de séjours
```

Hive traduit chaque requête HiveQL en une suite de jobs MapReduce : **chaque jointure ou GROUP BY ajoute un job**, avec un shuffle réseau et une écriture sur disque HDFS entre deux jobs. Une requête à 4 jointures enchaîne donc 4 à 5 jobs, ce qui explique le risque de dépasser les 2 minutes. Les moteurs Tez et Spark réduisent ce coût en exécutant tout le plan en mémoire.

- **5.5 Optimisation du modèle physique et mesures (1h45)** : Trois leviers ont été appliqués et mesurés :

Technique	Principe	Application au CHU
Partitionnement	Un sous-dossier par valeur ; un filtre ne lit que les dossiers utiles	Consultations et hospitalisations par `annee` ; décès et satisfaction par `annee, code_region`
Bucketing	N fichiers par hachage d'une colonne de jointure	16 buckets sur `sk_patient` (démonstration)
Vue matérialisée	Agrégat précalculé et stocké	Remplacée par le cache de l'API (10 min) et des tables d'agrégats possibles
Z-ORDER	Tri des fichiers pour sauter ceux hors filtre (min / max)	Établissement et diagnostic sur les consultations
Compactage	Regrouper les petits fichiers	`OPTIMIZE` sur toutes les tables Gold
Broadcast	Envoyer les petites dimensions à tous les nœuds	Jointures sans shuffle

```
CREATE TABLE fait_deces (id_deces STRING, sk_localisation BIGINT, age_deces INT,
...)
PARTITIONED BY (annee INT, code_region STRING)
CLUSTERED BY (sk_localisation) INTO 16 BUCKETS
STORED AS ORC;
SET hive.exec.dynamic.partition.mode = nonstrict;
INSERT OVERWRITE TABLE fait_deces PARTITION (annee, code_region) SELECT ... ;
CREATE MATERIALIZED VIEW mv_deces_region AS
  SELECT annee, code_region, COUNT(*) AS nb_deces FROM fait_deces GROUP BY annee,
code_region;
```

Résultats du benchmark (médiane de 5 exécutions, version naïve vs version optimisée) :

Requête	Avant	Après	Gain	Données lues
Q7 décès par région 2019 (25 M lignes) – Spark	244,2 s (CSV)	1,8 s	× 137	2,7 Gio → partitions 2019 seules
Q7 décès par région 2019 – Trino	1,78 s	0,42 s	× 4,3	110,7 → 13,8 Mio
Q1 consultation par établissement – Trino	1,57 s	0,56 s	× 2,8	52,2 → 11,1 Mio
Q2 consultation par diagnostic – Trino	0,84 s	0,36 s	× 2,3	36,5 → 1,0 Mio
Q3 taux global d'hospitalisation – Trino	0,90 s	0,44 s	× 2,1	36,2 → 4,9 Mio

Conclusion : toutes les analyses répondent en **moins de 0,7 s**, très loin du seuil de 2 minutes. Le gain dépend du volume : énorme sur les 25 M de décès, faible sur les petites tables (2 479 séjours) où le coût fixe de Delta (≈ 1 s) domine. Le bucketing supprime bien le shuffle dans le plan d'exécution, mais ne devient rentable qu'entre deux grandes tables ; le Z-ORDER le remplace en production.